

Università di Genova

Università degli Studi di Genova

Dipartimento di Ingegneria Navale, Elettrica, Elettronica e delle Telecomunicazioni

Corso di Studio in Ingegneria Elettronica e Tecnologie dell'Informazione

Tesi di Laurea Triennale

Ottobre 2026

Progettazione e sviluppo di un sistema IoT per il monitoraggio acustico in tempo reale

Design and development of an IoT system for real-time acoustic monitoring

Candidato
Luca Corsini

Relatore Prof. Riccardo Berta
Correlatore Dr. Matteo Fresta
Correlatore Prof. Luca Lazzaroni

Anno accademico 2025/2026

Sommario

Il presente lavoro di tesi illustra la progettazione e l'implementazione di un nodo IoT embedded per il monitoraggio acustico e vibrazionale in tempo reale. Il dispositivo integra un microcontrollore ESP32, un microfono MEMS digitale INMP441 gestito via interfaccia I^2S e un accelerometro triassiale ADXL362 connesso tramite bus SPI. L'elaborazione del segnale audio viene eseguita localmente tramite tecniche di DSP che includono la finestratura di Hamming, il calcolo della FFT e l'applicazione della curva di pesatura A nel dominio della frequenza. Sfruttando la struttura dual-core del microcontrollore, le operazioni di analisi spettrale e la gestione della connettività wireless vengono disaccoppiate per ridurre le interferenze tra elaborazione e comunicazione, senza implicare di per sé determinismo temporale. I dati elaborati sono trasmessi tramite protocollo WebSocket verso un'applicazione mobile sviluppata in framework Flutter per la visualizzazione dei livelli di pressione sonora, dello spettro e degli allarmi, integrando inoltre la sincronizzazione verso la piattaforma cloud Measurify. Le prove preliminari in ambienti di laboratorio e simulazione hanno verificato il funzionamento della catena di acquisizione, elaborazione e supervisione.

Indice

1 Introduzione **3**

2 Metodi e strumenti utilizzati **5**
2.1 Strumentazione hardware 5
2.2 Strumentazione software 6
2.2.1 Il framework Flutter e motivazioni della scelta 6
2.2.2 Architettura e moduli del codice 6
2.2.3 Analisi dei moduli di gestione ed elaborazione 7
2.2.4 Interfaccia utente 9
2.3 Schema a blocchi del sistema 10
2.4 Analisi spettrale tramite FFT 10

3 Sperimentazione e Risultati **12**
3.1 Applicazioni pratiche e valore applicativo del sistema 12
3.1.1 Monitoraggio acustico in eventi live e contesti teatrali 13
3.1.2 Rilevazione vibrazionale e supporto alla manutenzione 14
3.1.3 Monitoraggio integrato in sale, studi e spazi culturali 14
3.1.4 Confronto con soluzioni esistenti e vantaggi della proposta 15
3.1.5 Limiti applicativi e condizioni di impiego 15

4 Contributo personale e considerazioni conclusive **17**

Capitolo 1

Introduzione

Il presente lavoro di tesi ha come obiettivo la progettazione e lo sviluppo di un prototipo di sistema IoT per il monitoraggio acustico e vibrazionale in tempo reale. Il sistema è pensato come nodo embedded intelligente, capace di acquisire segnali digitali, elaborarli localmente tramite tecniche di DSP e trasmettere i dati attraverso una connessione Wi-Fi verso un'applicazione mobile sviluppata in Flutter.

L'architettura proposta è organizzata secondo una logica modulare, così da distinguere in modo chiaro le principali funzioni del sistema: acquisizione, elaborazione, comunicazione e supervisione. Questa scelta consente di semplificare la progettazione, migliorare la manutenibilità del codice e rendere il sistema facilmente estendibile in vista di futuri sviluppi.

Il progetto può essere collocato anche nell'ambito dell'*Internet of Musical Things* (IoMusT), in relazione ai possibili impieghi nell'audio dal vivo. Questa disciplina estende il paradigma dell'IoT al mondo della produzione e fruizione musicale, definendo i cosiddetti *Musical Things*: dispositivi embedded dotati di sensori e connettività capaci di acquisire, elaborare e scambiare dati a scopo acustico e musicale. In tale prospettiva, il nodo può fornire indicatori acustici e vibrazionali agli operatori, senza realizzare nell'attuale prototipo un controllo automatico dell'impianto audio.

Il nodo IoT realizzato integra un microcontrollore ESP32, scelto per la presenza di periferiche hardware dedicate alla comunicazione I^2S, per la connettività Wi-Fi integrata, per l'architettura dual-core e per il supporto nativo al sistema operativo in tempo reale FreeRTOS [2, 8]. A tali caratteristiche si aggiungono una potenza di calcolo sufficiente per la gestione di operazioni di elaborazione digitale in tempo reale ed un costo contenuto, rendendo il dispositivo adatto a un'applicazione embedded di monitoraggio continuo.

Il sistema utilizza due sensori digitali: il microfono MEMS InvenSense INMP441 e l'accelerometro ADXL362. Il primo consente l'acquisizione del segnale audio in formato PCM direttamente via interfaccia I^2S, riducendo la complessità della catena di acquisizione e limitando le problematiche legate alla conversione analogico-digitale. Il secondo permette invece la rilevazione delle vibrazioni strutturali lungo i tre assi spaziali, offrendo un'informazione complementare utile all'analisi del comportamento dinamico delle strutture d'allestimento.

La scelta di utilizzare sensori digitali e interfacce standard, quali I^2S e SPI, è coerente con l'obiettivo di realizzare un sistema compatto, affidabile e facilmente scalabile. Inoltre, questa impostazione riduce il numero di componenti analogici esterni e semplifica l'integrazione complessiva del nodo embedded.

L'elaborazione del segnale audio avviene direttamente sul dispositivo mediante una catena di processamento digitale che comprende la normalizzazione dei campioni, l'applicazione di una finestra di Hamming, il calcolo della FFT e l'estrazione dell'energia spettrale pesata secondo la curva di ponderazione A. In questo modo è possibile ottenere una stima del livello sonoro più coerente con la percezione uditiva umana, oltre a ricavare informazioni spettrali utili per l'analisi del segnale.

Per la supervisione remota, il sistema prevede una comunicazione wireless verso un'applicazione mobile cross-platform sviluppata con Flutter. L'applicazione consente la visualizzazione in tempo reale dei principali parametri rilevati dal nodo, tra cui il livello sonoro, le informazioni inerenti alle vibrazioni sugli assi X, Y e Z, lo stato di connessione e gli eventi di allarme. Inoltre, è presente un modulo di data logging che permette di salvare i dati in formato CSV per successive analisi offline.

Un ulteriore elemento dell'architettura è l'integrazione con una piattaforma cloud esterna, utilizzata come livello di raccolta e storicizzazione dei dati. L'app accumula i campioni in memoria e avvia un invio HTTP in formato JSON al raggiungimento di 100 campioni; è disponibile anche un invio manuale. L'effettivo salvataggio remoto richiede una risposta positiva del servizio.

Nel suo insieme, il progetto mira a dimostrare la fattibilità di una soluzione IoT e IoMusT capace di integrare acquisizione, elaborazione ed Edge-DSP in tempo reale. Il lavoro apre la strada a sviluppi futuri che spaziano dall'integrazione con protocolli audio di rete come Dante, fino alla creazione di scenari interattivi dal vivo in cui le misurazioni d'ambiente dialogano direttamente con gli impianti di diffusione e la strumentazione di palco.

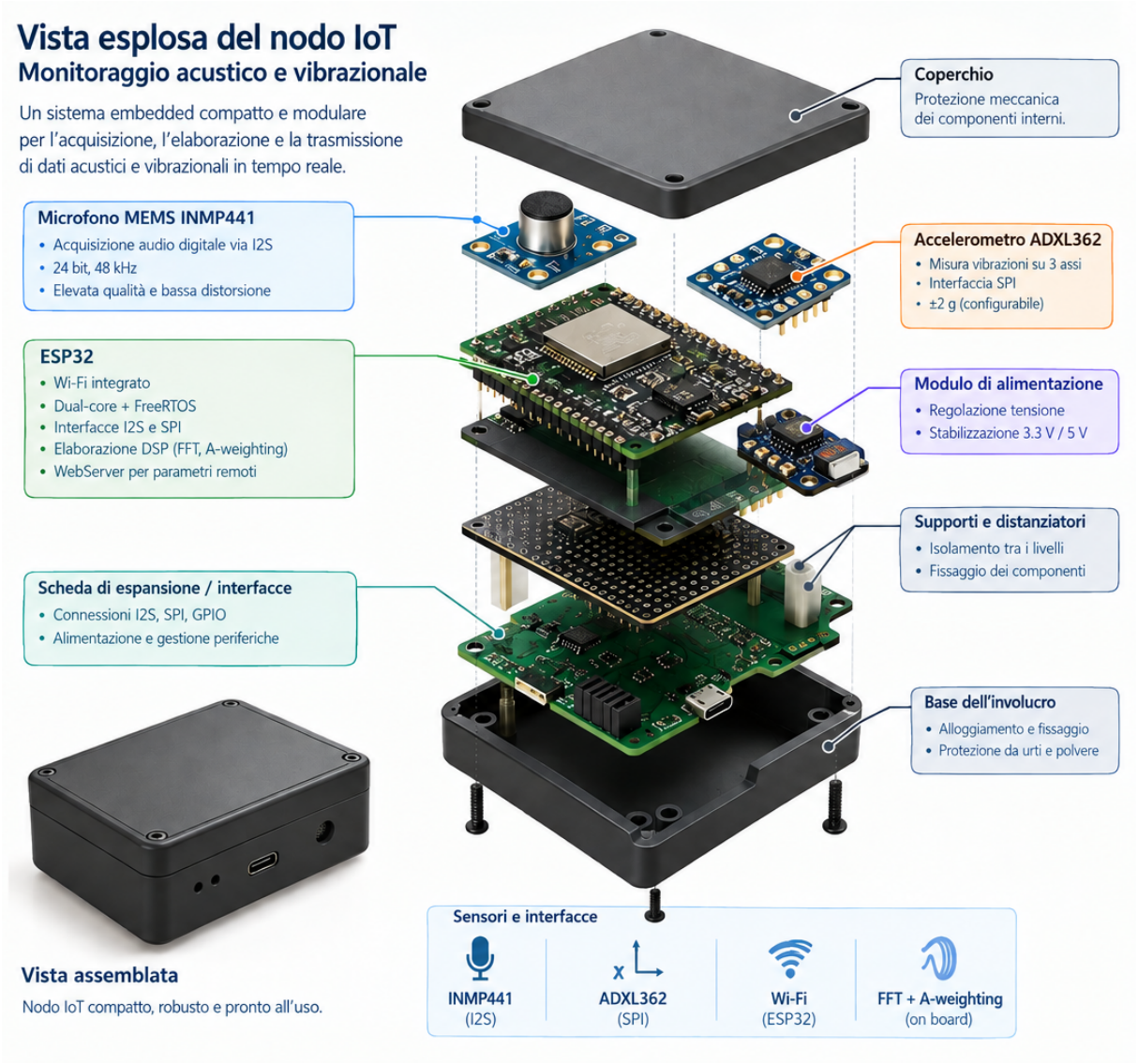


Figura 1.1: Vista esplosa del prototipo e schema a blocchi concettuale del sistema

Capitolo 2

Metodi e strumenti utilizzati

2.1 Strumentazione hardware

Per costruire il nodo IoT è stato scelto un microcontrollore ESP32 impiegato come piattaforma embedded per l'elaborazione digitale dei segnali. La scelta è stata motivata da differenti caratteristiche chiave:

- Presenza della periferica hardware I^2S integrata.
- Connettività Wi-Fi integrata (stack TCP/IP embedded).
- Architettura dual-core.
- Supporto nativo al sistema operativo FreeRTOS.
- Adeguata potenza di calcolo in virgola mobile (FPU).



Figura 2.1: ESP32

Insieme al DSP, il sistema integra due sensori digitali ad alte prestazioni: il microfono digitale InvenSense INMP441 e l'accelerometro ADXL362.

Il sensore TDK InvenSense INMP441 è un microfono MEMS digitale con interfaccia I^2S, progettato per applicazioni a bassa potenza che richiedono una buona qualità audio. La scelta di questo componente è motivata dai seguenti aspetti principali:

- **Interfaccia I^2S digitale:** il segnale audio è disponibile direttamente in formato PCM a 24 bit, permettendo il collegamento al microcontrollore senza un convertitore ADC esterno e riducendo quindi rumore, complessità circuitale e costi complessivi.
- **Prestazioni acustiche:** il microfono offre un rapporto segnale/rumore tipico di circa 61 dB e un punto di sovraccarico acustico (AOP) intorno a 120 dB SPL, valori adeguati anche per scenari con dinamica sonora relativamente elevata.
- **Basso consumo:** la corrente operativa dell'ordine del milliampere rende il dispositivo adatto a nodi IoT potenzialmente alimentati a batteria.
- **Ingombro ridotto:** le dimensioni contenute semplificano l'integrazione in sistemi embedded compatti per essere introdotti in qualsiasi ambiente.

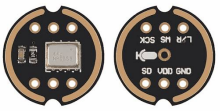


Figura 2.2: INMP441

Poiché il microfono fornisce campioni digitali via I^2S, è stato effettuato un confronto con un fonometro di riferimento per determinare un offset di calibrazione in dB. In assenza di un resoconto quantitativo delle prove, non viene attribuita una precisione metrologica alla stima. L'uscita digitale facilita inoltre l'elaborazione del segnale lato ESP32, consentendo l'applicazione di algoritmi di elaborazione nel dominio della frequenza e di filtri di pesatura A direttamente sui dati acquisiti, senza ulteriori conversioni analogico-digitale che potrebbero degradare il segnale.

Il sensore ADXL362 è un accelerometro MEMS triassiale a bassissimo consumo con interfaccia SPI, ottimizzato per applicazioni di monitoraggio continuo. Le principali ragioni della scelta sono:

- **Consumo ultra-low power:** il consumo energetico inferiore rispetto a molti accelerometri della stessa categoria lo rende adatto ad applicazioni IoT *always-on*, in quanto possiede funzionalità *event-driven wake-on-interrupt*.
- **Elevata risoluzione:** permette di rilevare vibrazioni di bassa ampiezza con buona precisione (1 mg/LSB nel range ±2g).
- **Modalità di interrupt integrata:** consente di attivare il sistema al superamento di determinate soglie, ottimizzando ulteriormente i consumi.
- **Stabilità termica:** favorisce misure affidabili in ambienti variabili.



Figura 2.3: ADXL362

L'adozione di interfacce digitali standard (I^2S e SPI) agevola l'integrazione dei sensori e possibili modifiche dell'architettura.

L'ESP32 è un microcontrollore dual-core, non un DSP dedicato. L'architettura prevede la separazione delle attività di acquisizione ed elaborazione da quelle di comunicazione; l'effettiva assegnazione dei task ai core e la gestione dei buffer DMA devono essere verificate nel firmware. Tale organizzazione mira a limitare le interferenze introdotte dalle operazioni di rete.

2.2 Strumentazione software

La componente applicativa è stata progettata per garantire l'acquisizione in tempo reale dei flussi dati ad alta frequenza trasmessi dal microcontrollore, l'elaborazione locale delle metriche di allarme, la visualizzazione grafica reattiva e l'interoperabilità con piattaforme cloud per la storicizzazione delle misure.

2.2.1 Il framework Flutter e motivazioni della scelta

L'applicazione di supervisione è stata sviluppata con Flutter e Dart per rendere accessibili da dispositivo mobile le misure acustiche e vibrazionali prodotte dal nodo ESP32. La scelta del framework risponde all'esigenza di disporre di un'interfaccia reattiva e di una base di codice multipiattaforma, capace di gestire un flusso continuo di dati e di presentare gli stati di allarme in modo immediato.

- Prestazioni grafiche ed esecuzione nativa: Flutter consente di comporre interfacce reattive senza ricorrere a una WebView. Nell'applicazione, i valori delle 16 bande spettrali sono mostrati mediante barre `AnimatedContainer`, mentre le accelerazioni sono rappresentate con indicatori di progresso. La fluidità dipende dal dispositivo e dalla frequenza effettiva dei pacchetti: il codice non garantisce un aggiornamento a 60 o 120 FPS.
- Gestione asincrona dei flussi dati: Le astrazioni `Future`, `Stream` e `Timer` sono utilizzate per ricevere i dati WebSocket, controllare i timeout, programmare le riconnessioni e inviare i campioni al cloud. Il controller notifica le modifiche dello stato all'interfaccia mediante `ChangeNotifier`.
- Architettura a componenti (Widget-based): La composizione mediante widget permette di suddividere la schermata nelle viste Home, FFT, Assi e Debug. La logica applicativa rimane nel controller e nei servizi, mentre la schermata si occupa della rappresentazione dei dati e dei comandi dell'utente.
- Multipiattaforma: Flutter consente di destinare la stessa base di codice a più piattaforme. Nel presente lavoro l'applicazione è stata provata su uno smartphone Android; non si attribuiscono ai test effettuati risultati sperimentali relativi a iOS.

2.2.2 Architettura e moduli del codice

Il software è organizzato in livelli di configurazione, servizi, controller e presentazione. `MonitoringController` estende `ChangeNotifier` e centralizza lo stato applicativo: è quindi più preciso descrivere la soluzione come un'architettura controller–service–view che come un'implementazione formale del pattern Model-View-Controller, in assenza di un livello Model separato.

1. Configuration Level (`AppConfig`): Contiene l'endpoint WebSocket, gli identificativi e il token necessari all'accesso al cloud, le soglie degli allarmi e i parametri temporali. La frequenza di campionamento audio è invece configurata nel firmware ESP32.

2. Service Level (`WebSocketService` e `MeasurifyService`): Gestisce rispettivamente la ricezione dei messaggi WebSocket e le richieste HTTP POST necessarie alla creazione della *measurement* e all'invio dei campioni.

3. Controller Level (`MonitoringController`): Riceve i dati interpretati dal servizio WebSocket, aggiorna le metriche, valuta gli allarmi, prepara i campioni da inviare al cloud e gestisce la registrazione locale.

4. Presentation Level (`DashboardScreen`): Visualizza i dati nelle quattro viste dell'applicazione e reagisce alle notifiche del controller.

2.2.3 Analisi dei moduli di gestione ed elaborazione

Gestione della connessione in tempo reale: `WebSocketService`

La comunicazione tra nodo ESP32 e applicazione mobile sfrutta il protocollo WebSocket [12, 23] mediante il package `web_socket_channel`, trasmettendo i dati strutturati in formato JSON [13]. Il servizio apre il canale, decodifica i messaggi JSON e notifica al controller la ricezione dei dati, gli errori e la chiusura della connessione. All'apertura del canale viene avviato un timer di sei secondi:

```
_handshakeTimer = Timer(const Duration(seconds: 6), () async {
  if (!_connected) {
    onError?.call('Timeout handshake WebSocket');
    await disconnect();
  }
});
```

La variabile `_connected` diventa vera alla ricezione del primo messaggio, che annulla il timer e richiama `onConnected`. Pertanto, nel codice, il timeout controlla l'arrivo del primo pacchetto e non soltanto il completamento della negoziazione WebSocket. L'eventuale assenza di dati successiva alla prima ricezione non è coperta da questo specifico timer.

Resilienza di rete e tolleranza alle disconnessioni temporanee

Per evitare che un'interruzione breve produca subito un allarme o modifichi ripetutamente l'interfaccia, il controller attende 1500 ms prima di passare dallo stato `connected` allo stato `reconnecting`. Se nel frattempo riceve un pacchetto, annulla il timer; altrimenti aggiorna lo stato e gli allarmi:

```
_toleranceTimer = Timer(
  const Duration(milliseconds: AppConfig.disconnectToleranceMs),
  () {
    _isToleratingDisconnect = false;
    connectionState = ConnectionStateView.reconnecting;
    handleReconnectionAttempt();
    _updateAlarms();
    notifyListeners();
  },
);
```

Durante tale intervallo, l'etichetta mostrata all'utente resta temporaneamente "Connesso". I tentativi successivi utilizzano ritardi temporizzati configurati nell'applicazione; non è implementata una strategia di attesa progressivamente crescente.

Algoritmo di rilevamento allarmi e identificazione del rientro acustico (effetto Larsen)

Il controller valuta i valori ricevuti e genera allarmi separati per connessione, livello sonoro e vibrazioni. Nel codice attuale l'avviso acustico si attiva a 75 dB(A) e l'allarme critico a 90 dB(A). La costante `criticalDbThreshold = 85.0` in `AppConfig` non è usata dal controller e va allineata prima della consegna del codice.

Per le vibrazioni si considera $a_{\max} = \max(|a_X|, |a_Y|, |a_Z|)$. Le soglie di attenzione e critica sono rispettivamente 20 e 28, espresse nella scala numerica ricevuta dal firmware. L'unità fisica non è verificabile dai sorgenti Flutter: non si presume una specifica conversione o calibrazione.

Il possibile effetto Larsen è segnalato solo con `currentDba` almeno pari a 70 dB(A) e almeno tre bande FFT positive; in tal caso viene applicata la seguente condizione:

```
return maxBand >= 45.0 &&
    average > 0 &&
    maxBand / average >= AppConfig.larsenPeakMultiplier;
```

Il moltiplicatore è impostato a 5 e la media comprende anche la banda dominante. In questa logica, `maxBand >= 45.0` rappresenta la soglia di ampiezza minima per identificare un picco anomalo. Si tratta di un criterio euristico per segnalare una concentrazione spettrale anomala, non di una prova univoca della presenza di feedback acustico; la scala numerica delle bande è definita dai dati trasmessi dal firmware.

Integrazione con la piattaforma cloud REST Measurify

Il controller prepara i dati per Measurify come campioni `TimeserieSample`, ciascuno con timestamp espresso come stringa di millisecondi dall'epoca Unix e con un vettore di 20 valori:

1. Elemento 0: valore ricevuto nel campo `dBA`.
2. Elementi 1–3: valori accelerometrici X, Y e Z.
3. Elementi 4–19: 16 valori FFT; prima dell'invio, la lista viene completata con zeri o troncata se necessario.

Il singolo campione viene serializzato come segue:

```
Map<String, dynamic> toJson() => {
  'timestamp': timestamp,
  'values': values,
};
```

I campioni vengono accumulati in memoria e l'invio automatico è avviato quando il buffer raggiunge 100 elementi. Il pulsante "Invia a Measurify" aggiunge un campione corrente e forza l'invio dei dati presenti nel buffer. Il servizio richiede prima la creazione della *measurement*, se non risulta già inizializzata nella sessione, quindi invia l'array dei campioni tramite HTTP POST a `/v1/measurements/{measurementId}/timeserie`, secondo la configurazione del servizio applicativo e la documentazione del repository [26]. L'autenticazione usa un token nell'header `Authorization`, il cui valore non è riportato nell'elaborato.

Logging locale ed esportazione dati in formato CSV

Quando l'utente avvia una registrazione, il controller aggiunge a un buffer locale una riga per ogni pacchetto ricevuto, con timestamp, livello sonoro, assi accelerometrici e valori FFT. `Stopwatch` misura la durata della sessione, mentre `Timer.periodic` aggiorna l'indicazione grafica ogni secondo: non imposta la frequenza di acquisizione. Al termine, il buffer viene scritto in un file CSV temporaneo e condiviso tramite `share_plus`:

```
final csv = StringBuffer('timestamp,dBA,accX,accY,accZ,fft\n');
for (final row in csvRows) {
  csv.writeln(row.join(','));
}
await file.writeAsString(csv.toString());
await Share.shareXFiles([XFile(file.path)]);
```

2.2.4 Interfaccia utente

La navigazione utilizza `NavigationBar` e comprende quattro viste:

- Dashboard: Riporta stato WebSocket, livello sonoro, valori accelerometrici, allarmi ordinati per priorità e contatori della sessione. Consente inoltre l'invio manuale a Measurify e l'avvio o l'arresto della registrazione CSV.
- Analisi Spettrale FFT: Mostra i valori ricevuti mediante barre `AnimatedContainer` e intervalli di frequenza indicativi. È una visualizzazione a barre aggiornata nel tempo, non uno spettrogramma.
- Analisi Vettoriale: Presenta le componenti X, Y e Z con indicatori calcolati dal valore assoluto dell'asse, normalizzato rispetto a 30.
- Pannello di diagnostica: Visualizza stato ed endpoint della connessione, tentativi di riconnessione, contatori, ultimo errore e ultimo pacchetto ricevuto. Include un elenco esplicativo di possibili problemi, non uno storico automatico degli errori.

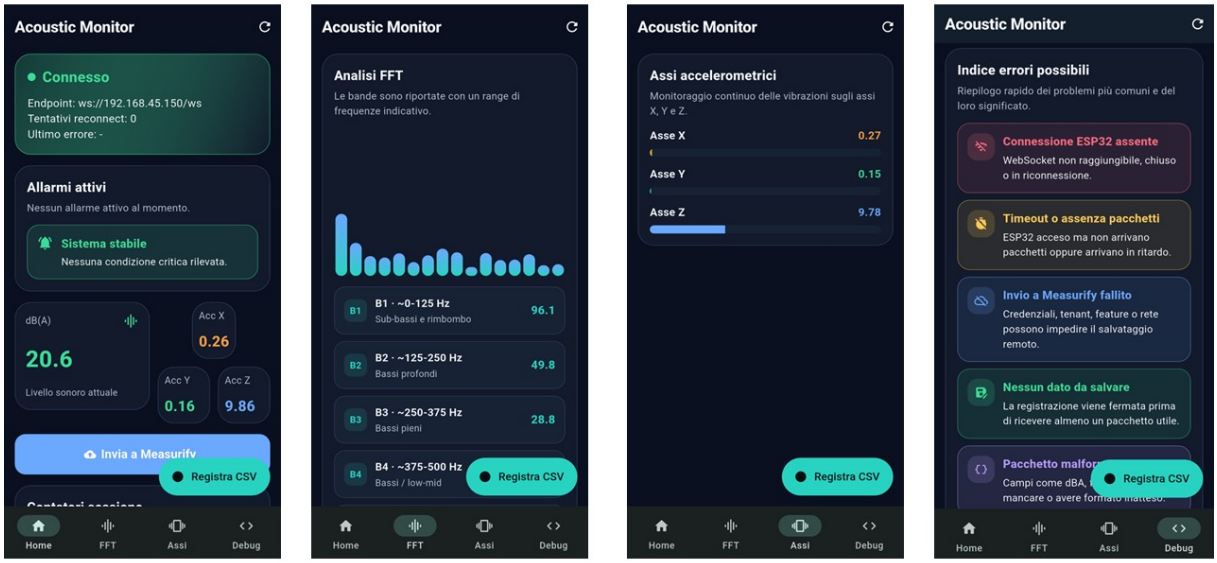


Figura 2.4: Interfaccia UI dell'applicazione mobile

2.3 Schema a blocchi del sistema

La Figura 2.5 mostra il diagramma a blocchi dell'architettura complessiva del sistema.

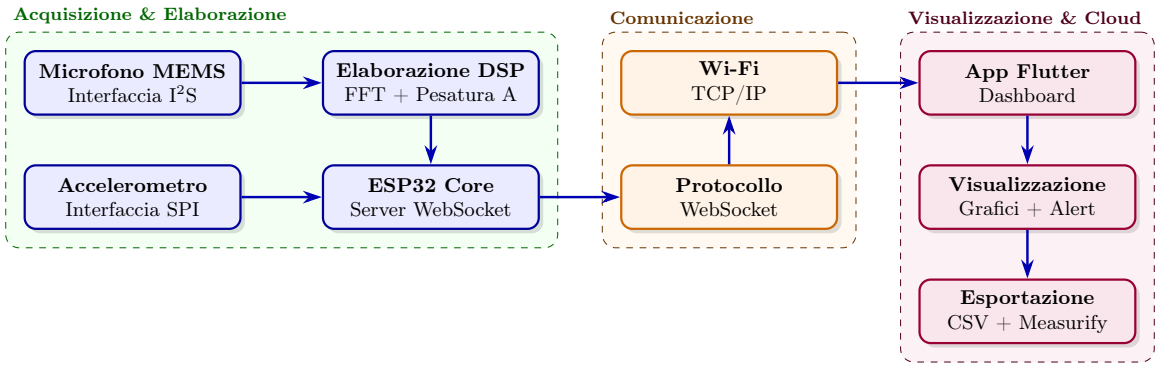


Figura 2.5: Diagramma a blocchi del sistema IoT di monitoraggio acustico

2.4 Analisi spettrale tramite FFT

La frequenza di campionamento prevista è $f_s = 48\,\text{kHz}$ in quanto rappresenta uno standard consolidato in ambito audio digitale e consente di coprire l'intero spettro udibile (20 Hz – 20 kHz) nel rispetto del teorema di Nyquist, garantendo un'adeguata banda utile per il monitoraggio acustico ambientale.

L'analisi nel dominio della frequenza è descritta mediante FFT su finestre di $N = 1024$ campioni; frequenza e dimensione effettivamente impostate devono essere verificate nel firmware. La scelta della dimensione della finestra rappresenta un compromesso tra: risoluzione in frequenza, latenza temporale, carico computazionale e occupazione di banda per l'invio dei dati tramite Wi-Fi.

La risoluzione in frequenza risulta:

$$\Delta f = \frac{f_s}{N} = \frac{48.000}{1024} \approx 46.9\,\text{Hz}$$

Il tempo di acquisizione della finestra è:

$$T_{\text{acq}} = \frac{N}{f_s} = \frac{1024}{48.000} \approx 21.3\,\text{ms}$$

Il tempo di acquisizione di circa 21 ms riguarda una finestra di campioni e non coincide necessariamente con la latenza complessiva di elaborazione, trasmissione e visualizzazione.

Prima dell'applicazione della FFT, ai campioni acquisiti viene applicata una finestra di Hamming. La scelta di tale finestra deriva dalla necessità di ridurre il leakage spettrale dovuto alla troncatura del segnale nel dominio temporale [14, 17].

In assenza di finestratura, la discontinuità ai bordi della finestra introdurrebbe componenti rumorose nello spettro, compromettendo l'accuratezza della stima energetica. La finestra di Hamming rappresenta un buon compromesso tra attenuazione dei lobi laterali e larghezza del lobo principale, risultando adeguata per analisi energetiche a banda larga.

L'elaborazione digitale segue la sequenza:

1. Normalizzazione dei campioni acquisiti (24 bit effettivi) per garantire coerenza numerica nell'elaborazione floating-point;
2. Applicazione della finestra di Hamming;
3. Calcolo della FFT forward;
4. Conversione dei coefficienti complessi in modulo.

L'analisi viene limitata ai primi $N/2$ bin, in quanto per segnali reali lo spettro risulta simmetrico e, in accordo con il teorema di Nyquist, le componenti oltre $f_s/2$ non contengono informazione aggiuntiva. L'app Flutter riceve una rappresentazione di 16 valori spettrali e la visualizza con etichette di frequenza

indicative. La modalità di aggregazione dei bin sul nodo e la corrispondenza tra valori ed etichette richiedono una verifica nel firmware.

La pesatura A consente di costruire un indicatore che tiene conto della sensibilità uditiva in funzione della frequenza; il riferimento allo standard IEC 61672-1 non implica che il prototipo sia un fonometro conforme [18]. L'effettiva implementazione nel dominio della frequenza va verificata nel firmware.

L'energia spettrale pesata può essere espressa, prima dei fattori di normalizzazione e calibrazione, come:

$$E = \sum_{i=1}^{N/2} (|X_i| \cdot A_i)^2$$

dove A_i è il coefficiente lineare di pesatura A per il bin i. Per ricavare un livello assoluto in dB(A) occorrono la corretta normalizzazione spettrale, la compensazione della finestra e la calibrazione.

L'applicazione della pesatura nel dominio spettrale consente maggiore flessibilità rispetto a un filtraggio analogico, riduce la complessità hardware e permette eventuali modifiche future (ad esempio pesature C o Z) senza interventi sul front-end di acquisizione.

Capitolo 3

Sperimentazione e Risultati

La fase di collaudo dell'applicazione mobile è stata condotta collegando uno smartphone Android in modalità di debug USB all'ambiente di sviluppo Visual Studio Code. Tale configurazione ha consentito la build e l'esecuzione diretta del codice Dart/Flutter sul dispositivo fisico, verificando le prestazioni dell'interfaccia utente in condizioni di lavoro reali.

Una volta completato il codice sia del firmware sull'ESP32 sia dell'applicazione Flutter, il passo successivo è stato la realizzazione effettiva del prototipo hardware e la relativa fase di prove sul campo.

Per il collegamento dei componenti sono stati utilizzati cavi Dupont schermati e flessibili, collegando il microfono MEMS INMP441 all'interfaccia I^2S e l'accelerometro ADXL362 al bus SPI dell'ESP32. I connettori rimovibili agevolano la sostituzione dei componenti durante la prototipazione, ma richiedono attenzione alla stabilità dei contatti e all'immunità ai disturbi in un'installazione operativa.

Dopo l'assemblaggio sono state effettuate prove funzionali; i risultati numerici e il protocollo delle prove restano da documentare per una validazione quantitativa:

- **Calibrazione acustica e livello sonoro:** Le letture iniziali dell'INMP441 non corrispondevano direttamente ai livelli indicati dal fonometro di riferimento. In assenza di un fonometro certificato, non è possibile quantificare l'incertezza della stima.
- **Rilevazione delle vibrazioni e soglie d'allarme:** Durante i test dell'accelerometro ADXL362 si è notato che piccole vibrazioni ambientali trasmesse dalla superficie di appoggio potevano generare falsi positivi negli allarmi. Nel controller Flutter sono presenti soglie fisse di attenzione e allarme.
- **Streaming WebSocket e resa grafica:** La ricezione dei pacchetti JSON ha consentito la visualizzazione dei valori e delle barre corrispondenti alle 16 bande; la frequenza effettiva del flusso e la latenza dell'interfaccia richiedono misure dedicate.

3.1 Applicazioni pratiche e valore applicativo del sistema

Il prototipo sviluppato è concepito come nodo IoT per il monitoraggio integrato di grandezze acustiche e vibrazionali. L'architettura proposta combina l'acquisizione del segnale audio mediante microfono MEMS digitale, la rilevazione delle accelerazioni sui tre assi spaziali, l'elaborazione locale dei dati e la loro trasmissione a un'applicazione di supervisione. Questa integrazione consente di utilizzare il sistema in contesti nei quali la qualità sonora, le condizioni meccaniche dell'ambiente e la disponibilità di informazioni in tempo reale rappresentano aspetti rilevanti per l'operatore.

Il nodo non è concepito come sostituto di un fonometro certificato né come strumento di verifica strutturale. Esso rappresenta tuttavia un sistema di monitoraggio distribuito a basso costo, in grado di fornire indicatori acustici e vibrazionali continui, segnalare variazioni anomale e supportare decisioni operative, interventi manutentivi o ulteriori approfondimenti tecnici. La misura acustica richiede inoltre una procedura di calibrazione rispetto a uno strumento di riferimento, poiché i microfoni MEMS a basso costo presentano una risposta che può variare rispetto a strumenti metrologici professionali [5, 18].

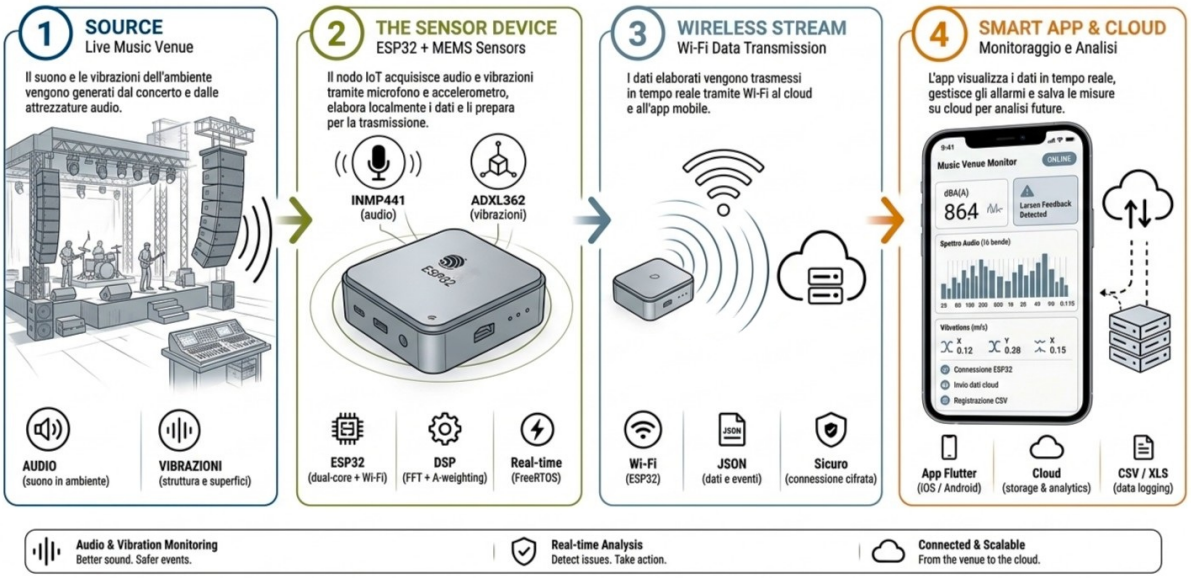


Figura 3.1: Schema grafico del comportamento pratico del sistema

3.1.1 Monitoraggio acustico in eventi live e contesti teatrali

Un primo scenario applicativo riguarda gli eventi musicali dal vivo, i teatri, gli auditorium e le installazioni temporanee. In questi contesti, il livello sonoro può variare rapidamente in funzione del programma musicale, del numero di sorgenti attive, della configurazione dell'impianto di diffusione, del riempimento della sala e della posizione dell'ascoltatore.

Il nodo proposto acquisisce il segnale dal microfono INMP441 e ne esegue l'elaborazione direttamente sull'ESP32. Attraverso il calcolo della FFT e l'applicazione della pesatura A, il sistema ricava un indicatore sintetico del livello acustico e una rappresentazione semplificata del contenuto spettrale. L'operatore può osservare i valori correnti, le barre spettrali e gli allarmi e, se necessario, registrare i pacchetti ricevuti in un file CSV.

La rappresentazione delle bande permette di osservare la concentrazione dell'energia spettrale. La logica di allarme per un possibile feedback è euristica e richiede la valutazione dell'operatore; il codice non implementa un riconoscimento autonomo di risonanze ambientali o accumuli alle basse frequenze. Tale funzionalità non intende sostituire le valutazioni condotte mediante analizzatori di livello professionale o software di misura dedicati, bensì fornire un indicatore ausiliario per il monitoraggio continuo, facilitando tempestivi interventi correttivi durante l'evento.

La Figura 3.2 mostra un possibile scenario di installazione del nodo su una struttura di supporto. Il posizionamento del dispositivo in prossimità dell'area da monitorare permette di acquisire informazioni correlate sia al campo sonoro locale sia alle vibrazioni trasmesse alla struttura.



Figura 3.2: Esempio concettuale di installazione del nodo IoT su una struttura truss in ambiente live. Il sistema acquisisce grandezze acustiche e vibrazionali e trasmette gli indicatori elaborati verso l'interfaccia di supervisione mobile.

3.1.2 Rilevazione vibrazionale e supporto alla manutenzione

Il secondo ambito applicativo riguarda il rilevamento delle vibrazioni mediante l'accelerometro triassiale ADXL362. Il sensore acquisisce dati sui tre assi X, Y e Z; l'interpretazione fisica dei valori presentati nell'app richiede la verifica della scala impostata nel firmware [6].

In ambienti di spettacolo o contesti tecnici temporanei, il nodo può essere vincolato a elementi quali truss, supporti per diffusori, strutture di scena o involucri di apparecchiature elettromeccaniche. La misura delle accelerazioni triassiali consente di rilevare variazioni rispetto al comportamento dinamico ordinario osservato durante il funzionamento, offrendo un indicatore relativo da confrontare nel tempo; non sono documentate prove su strutture reali.

L'uso dell'accelerometro non deve essere interpretato come sostituzione di una verifica ingegneristica strutturale. Il sistema non è progettato per certificare la sicurezza di strutture, carichi sospesi o dispositivi di sollevamento. Il suo contributo consiste nel fornire un monitoraggio continuo e a basso impatto, utile per individuare condizioni anomale, confrontare misure raccolte in istanti differenti e supportare attività di manutenzione preventiva.

L'ADXL362 risulta particolarmente adatto a questo tipo di architettura grazie al ridotto consumo energetico e alla disponibilità di funzionalità event-driven. Queste caratteristiche consentono, in prospettiva, di attivare procedure di acquisizione o di invio dati più frequenti soltanto in presenza di vibrazioni superiori a soglie prefissate [6].

3.1.3 Monitoraggio integrato in sale, studi e spazi culturali

Un terzo caso d'uso riguarda sale prova, studi di registrazione, auditorium, spazi espositivi, teatri storici e ambienti nei quali la qualità acustica e le sollecitazioni meccaniche possono essere osservate in modo congiunto. Un futuro impiego di più nodi distribuiti potrebbe raccogliere misure in punti diversi dello stesso ambiente, confrontando le variazioni del livello sonoro e delle vibrazioni locali.

In uno studio di registrazione, ad esempio, il nodo può essere utilizzato per osservare la presenza di rumore ambientale, risonanze persistenti o vibrazioni trasmesse a pareti, supporti e strutture leggere. In un teatro storico, una rete di nodi potrebbe invece contribuire alla raccolta di dati durante eventi musicali, consentendo di correlare le condizioni acustiche interne con le vibrazioni rilevate in specifici punti dell'edificio.

L'integrazione tra analisi acustica e vibrazionale costituisce uno degli elementi distintivi della soluzione proposta. Molti sistemi di monitoraggio a basso costo sono infatti orientati esclusivamente alla misura del rumore oppure alla sola acquisizione di dati ambientali. Il progetto riunisce i due flussi di dati nell'applicazione; la rete distribuita di più nodi resta una prospettiva futura. Ciò permette all'operatore di analizzare in modo coordinato gli eventi, senza utilizzare strumenti indipendenti per la supervisione di parametri strettamente correlati al contesto operativo.

3.1.4 Confronto con soluzioni esistenti e vantaggi della proposta

Le soluzioni disponibili sul mercato per il monitoraggio acustico professionale comprendono fonometri certificati, analizzatori in tempo reale, sistemi di monitoraggio continuo e piattaforme dedicate alla misura ambientale. A seconda del modello e delle procedure di taratura, tali strumenti possono offrire caratteristiche metrologiche e funzionalità specialistiche documentate. Tuttavia, il loro costo, la necessità di personale specializzato e la minore flessibilità di integrazione possono limitarne l'impiego in installazioni sperimentali, didattiche o distribuite.

Il progetto sviluppato non intende competere direttamente con tali strumenti sul piano della certificazione metrologica. L'obiettivo è piuttosto fornire una piattaforma modulare ed economicamente accessibile per il monitoraggio operativo. La stima dei soli componenti del nodo è riportata nella Tabella 3.2; il costo di una rete completa richiederebbe una valutazione separata.

L'ESP32 integra acquisizione digitale, elaborazione locale e connettività Wi-Fi. L'app riceve indicatori numerici anziché audio grezzo, riducendo il traffico rispetto allo streaming PCM continuo; gli allarmi visualizzati sono calcolati nel controller Flutter.

La Tabella 3.1 confronta qualitativamente il prototipo con la strumentazione professionale certificata; non rappresenta una comparazione sperimentale tra prodotti specifici.

Caratteristica	**Strumentazione professionale certificata**	**Prototipo IoT sviluppato**
Finalità principale	Misura metrologica, verifica normativa e campagne di misura	Monitoraggio operativo, supervisione remota e sperimentazione
Costo per punto di misura	Generalmente elevato	Stimato dai componenti; non comprende sviluppo e validazione
Misura acustica	Conformità dipendente dallo strumento e dalle procedure adottate	Indicativa, soggetta a calibrazione e validazione
Analisi vibrazionale integrata	Spesso disponibile solo mediante strumenti dedicati	Integrata tramite accelerometro triassiale
Elaborazione locale	Dipendente dal modello e dalla configurazione	FFT e pesatura A previste sul nodo; soglie di allarme anche nell'app
Connettività e dashboard	Variabile, spesso proprietaria	Wi-Fi/WebSocket verso Flutter; HTTP REST verso Measurify
Scalabilità	Dipendente dal numero e dal costo degli strumenti	Possibile estensione a più nodi, non validata sperimentalmente

Tabella 3.1: Confronto qualitativo tra il prototipo sviluppato e differenti classi di soluzioni per il monitoraggio acustico.

Il progetto fornisce una base modulare per un'eventuale rete di nodi; l'interoperabilità con ulteriori sensori e infrastrutture resta da progettare e validare. Una futura rete potrebbe affiancare gli strumenti certificati nel monitoraggio operativo, senza sostituirli nelle verifiche normative.

3.1.5 Limiti applicativi e condizioni di impiego

L'impiego del prototipo richiede alcune considerazioni. In primo luogo, l'accuratezza del livello sonoro dipende dall'eventuale calibrazione documentata, dalla risposta in frequenza del microfono, dal contenitore utilizzato, dalla posizione di installazione e dalle condizioni ambientali. Di conseguenza, i valori ottenuti devono essere interpretati come indicatori per il monitoraggio, soggetti a calibrazione e non come misure certificabili ai fini normativi.

In secondo luogo, l'analisi FFT a 16 bande fornisce una rappresentazione compatta dello spettro e risulta adeguata alla supervisione in tempo reale, ma non sostituisce un'analisi a bande di ottava, di terzo d'ottava o una misura ad alta risoluzione effettuata con strumentazione specialistica.

Infine, i dati vibrazionali consentono di individuare variazioni relative del comportamento dinamico nel punto di misura, ma non permettono di eseguire una diagnosi strutturale completa. Ogni condizione

di allarme deve quindi essere interpretata come richiesta di verifica tecnica, e non come valutazione definitiva dello stato di sicurezza della struttura.

Nonostante tali limiti, il sistema costituisce una base concreta per sviluppare un'infrastruttura di monitoraggio distribuita, adattabile a contesti audio professionali, culturali e sperimentali. L'evoluzione futura verso più nodi sincronizzati, algoritmi di classificazione degli eventi, integrazione con protocolli audio professionali e procedure di calibrazione più strutturate potrà incrementarne ulteriormente l'utilità applicativa.

La Tabella 3.2 riporta una stima orientativa dei soli componenti per un nodo; non comprende manodopera, spedizione, strumenti di misura o costi di sviluppo.

Componente	**Costo indicativo [euro]**
Scheda di sviluppo ESP32	8,00
Microfono MEMS INMP441	2,50
Accelerometro ADXL362	7,00
Cavetteria, connettori e prototipazione	3,00
Alimentazione e contenitore	9,00
Totale componenti per nodo	**29,50**

Tabella 3.2: Stima orientativa dei componenti per un nodo, in euro; prezzi da documentare con preventivi o ricevute.

Capitolo 4

Contributo personale e considerazioni conclusive

L'idea alla base di questo lavoro di tesi nasce dall'interesse per l'ingegneria del suono, maturato durante il percorso di studi in ingegneria elettronica e tecnologie dell'informazione e consolidato attraverso l'esperienza professionale come fonico. L'attività svolta in contesti internazionali e in città quali Amsterdam, Madrid, Monaco e Londra mi ha permesso di osservare direttamente le problematiche associate alla gestione dell'acustica e al controllo dei sistemi di diffusione sonora durante eventi complessi.

Il progetto è stato sviluppato a partire dall'integrazione tra le competenze acquisite nell'ambito dell'elaborazione digitale dei segnali, dei sistemi embedded e delle tecnologie IoT, e le esigenze operative riscontrate nel settore dell'audio dal vivo. In quest'ottica, il sistema non è stato concepito esclusivamente come una dimostrazione teorica, ma come il prototipo di una possibile rete distribuita di nodi sensoristici destinata ad ambienti quali teatri, auditorium e palchi per concerti.

Il contributo personale ha riguardato la definizione dell'architettura del sistema, l'implementazione della catena di acquisizione ed elaborazione del segnale e lo sviluppo delle funzionalità di comunicazione con i livelli di supervisione. In particolare, il nodo Edge-DSP è stato progettato per eseguire localmente l'analisi spettrale del segnale e per trasmettere i principali indicatori a un'interfaccia di supervisione e a una piattaforma remota. L'elaborazione locale consente di ridurre la quantità di dati da trasferire e di ottenere un riscontro con latenza contenuta, mantenendo al tempo stesso la possibilità di archiviare e consultare le informazioni acquisite.

Un ulteriore elemento caratterizzante del progetto è l'integrazione del monitoraggio acustico con l'analisi delle vibrazioni. Questa funzionalità può fornire un supporto strumentale all'osservazione del comportamento di strutture temporanee o storiche durante eventi caratterizzati da elevati livelli di pressione sonora o da condizioni potenzialmente sensibili alle vibrazioni. Il sistema non intende sostituire le verifiche strutturali previste dalla normativa e dalle procedure di sicurezza, ma può costituire uno strumento complementare per l'individuazione di variazioni o condizioni anomale da sottoporre a ulteriori valutazioni.

L'implementazione descritta mostra la fattibilità di un nodo compatto con elaborazione locale e supervisione mobile; la disponibilità della storicizzazione remota dipende dall'esito delle richieste al servizio cloud. Le prove descritte sono principalmente funzionali; accuratezza, ripetibilità e comportamento in diversi ambienti richiedono dati sperimentali più estesi.

Il sistema presenta tuttavia alcuni limiti, principalmente legati all'estensione della campagna sperimentale, al numero di ambienti analizzati e alla necessità di validare ulteriormente le prestazioni in condizioni operative più eterogenee. In particolare, l'integrazione con infrastrutture audio professionali e l'automazione delle azioni di controllo richiederebbero ulteriori attività di progettazione, verifica e validazione.

Sviluppi futuri

L'architettura modulare implementata pone le basi per diverse evoluzioni, orientate all'integrazione del sistema con gli standard e i flussi di lavoro dell'audio professionale.

- **Integrazione con il protocollo Dante®:** uno sviluppo prioritario consiste nell'aggiunta della compatibilità con Dante, uno dei protocolli più diffusi per la distribuzione di segnali audio su rete IP nel settore professionale. Tale integrazione permetterebbe al nodo di interfacciarsi direttamente con

l'infrastruttura audio dell'evento, riducendo la necessità di conversioni intermedie e semplificando l'accesso ai segnali disponibili sulla rete.

- **Riduzione automatica del rischio di feedback:** l'algoritmo di rilevazione dell'effetto Larsen potrebbe essere approfondito e, previa validazione, collegato alle console di mixaggio attraverso protocolli quali OSC o MIDI su IP. Il sistema potrebbe quindi suggerire o applicare filtri notch dinamici sulle frequenze interessate, previa definizione di opportuni criteri di sicurezza e di supervisione da parte dell'operatore.
- **Ottimizzazione acustica assistita:** le funzionalità di elaborazione locale potrebbero essere sviluppate verso algoritmi capaci di analizzare nel tempo l'energia spettrale e la risposta dell'ambiente, fornendo indicazioni sulle possibili correzioni di equalizzazione e sull'allineamento temporale dei diffusori. In una fase successiva, tali indicazioni potrebbero essere integrate in procedure di ottimizzazione semiautomatica o automatica, da validare mediante campagne di misura dedicate.
- **Estensione della rete sensoristica:** l'impiego di più nodi distribuiti consentirebbe di confrontare le misure acustiche e vibrazionali provenienti da punti differenti dell'ambiente, migliorando la capacità di localizzare fenomeni anomali e di correlare le condizioni operative dell'impianto con la risposta dello spazio e delle strutture.

In conclusione, il progetto costituisce un collegamento concreto tra l'elaborazione digitale dei segnali, i sistemi embedded e le applicazioni professionali nell'ambito dell'audio dal vivo. Il prototipo costituisce una piattaforma sperimentale che integra indicatori acustici e vibrazionali e segnala condizioni anomale sulla base di soglie configurate. Gli sviluppi futuri potranno trasformare questa architettura in un ecosistema IoT distribuito, destinato a supportare gli operatori nella valutazione della qualità sonora e nell'osservazione delle condizioni operative di impianti e strutture durante eventi complessi.

Bibliografia

[1] Espressif Systems, *ESP32 Technical Reference Manual*, https://documentation.espressif.com/esp32_technical_reference_manual_en.html.

[2] Espressif Systems, *ESP32 Series Datasheet*, https://documentation.espressif.com/esp32_datasheet_en.html.

[3] Espressif Systems, *ESP-IDF Programming Guide*, https://docs.espressif.com/projects/esp-idf/en/latest/esp32/.

[4] Espressif Systems, *ESP32 Hardware Design Guidelines*, https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32/.

[5] InvenSense, TDK, *INMP441: Omnidirectional Microphone with Bottom Port and Digital Output – Data Sheet*, https://product.tdk.com/system/files/dam/doc/product/sw_piezo/mic/mems-mic/data_sheet/inmp441.pdf.

[6] Analog Devices, *ADXL362: Micropower, 3-Axis, ±2 g/±4 g/±8 g Digital Output MEMS Accelerometer – Data Sheet*, https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL362.pdf.

[7] Analog Devices, *ADXL362: Product Information*, https://www.analog.com/en/products/adxl362.html.

[8] FreeRTOS, *FreeRTOS Kernel Documentation*, https://www.freertos.org/Documentation/00-Overview.

[9] Google, *Flutter Documentation*, https://docs.flutter.dev/.

[10] Google, *Flutter Cookbook: Communicate with WebSockets*, https://docs.flutter.dev/cookbook/networking/web-sockets.

[11] Dart Team, *WebSocket Class – dart:io API Reference*, https://api.dart.dev/dart-io/WebSocket-class.html.

[12] IETF, *RFC 6455: The WebSocket Protocol*, 2011, https://datatracker.ietf.org/doc/html/rfc6455.

[13] T. Bray, Ed., *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format*, Internet Engineering Task Force, 2017, https://datatracker.ietf.org/doc/html/rfc8259.

[14] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Prentice Hall, 2009.

[15] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007.

[16] R. B. Blackman and J. W. Tukey, *The Measurement of Power Spectra from the Point of View of Communications Engineering*, Dover Publications, 1958.

[17] F. J. Harris, "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978, doi: https://doi.org/10.1109/PROC.1978.10837.

[18] International Electrotechnical Commission, *IEC 61672-1:2013 – Electroacoustics – Sound Level Meters – Part 1: Specifications*, 2nd ed., 2013, https://webstore.ansi.org/standards/iec/iec61672ed2013.

[19] International Electrotechnical Commission, *IEC 61672-2:2013 – Electroacoustics – Sound Level Meters – Part 2: Pattern Evaluation Tests*, 2013.

[20] International Electrotechnical Commission, *IEC 60942:2017 – Electroacoustics – Sound Calibrators*, 2017.

[21] International Organization for Standardization, *ISO 1996-1:2016 – Acoustics – Description, Measurement and Assessment of Environmental Noise – Part 1: Basic Quantities and Assessment Procedures*, 2016.

[22] International Organization for Standardization, *ISO 1996-2:2017 – Acoustics – Description, Measurement and Assessment of Environmental Noise – Part 2: Determination of Sound Pressure Levels*, 2017.

[23] N. Mitrović, M. ĐorĐević, S. Veljković, and D. Danković, "Implementation and Testing of WebSocket Protocol in ESP32-Based IoT Systems," *Facta Universitatis, Series: Electronics and Energetics*, vol. 36, no. 2, pp. 267–284, 2023, doi: https://doi.org/10.2298/FUEE2302267M.

[24] R. Berta, A. De Gloria et al., *Measurify: documentazione e repository del progetto*, DITEN, Università degli Studi di Genova, https://github.com/measurify.

[25] Measurify, *Measurify: Cloud-Based Measurement-Oriented API*, https://github.com/measurify/measurify.github.io.

[26] Measurify, *Measurify Server: RESTful API*, https://github.com/measurify/server.

[27] Measurify, *Measurify Edge Library*, https://github.com/measurify/edge.

[28] Andrea Corzani, *Progettazione e realizzazione di uno smart distributed system per il controllo degli accessi in ambito aziendale*, Tesi di Laurea Magistrale in Ingegneria e Scienze Informatiche, Alma Mater Studiorum - Università di Bologna, Campus di Cesena, 2013.

[29] E. Botella Pastor, *Design of a Nurse Calling System with Real Time Indoor Locations Capabilities*, Technical Report, Universitat Politècnica de Catalunya, Escola Superior d'Enginyeries Industrial, Aerospacial i Audiovisual de Terrassa, 2023.

Ringraziamenti

Ai miei genitori e a mio fratello
Che con pazienza e incrollabile fiducia hanno creduto in me lungo tutto questo percorso, anche nei momenti di maggiore incertezza. Grazie per avermi dato la libertà di scegliere e la forza di affrontare i passaggi più complessi, insegnandomi il valore della responsabilità senza mai farmi sentire solo. Ogni traguardo di oggi porta con sé il vostro silenzioso e fondamentale sostegno.

A Debora
Che è comparsa dal nulla proprio quando ne avevo più bisogno, ed è stata capace di stravolgere la mia vita e riempirla di una gioia immensa. Mi hai insegnato a essere felice ed ansioso allo stesso tempo, e mi hai mostrato che si può essere fieri di se stessi anche di fronte ai fallimenti. Quello che mi hai donato ha sfumature di colore così vive da non poter essere racchiuso nelle parole di questa tesi.

Ai miei amici Alice, Edoardo, Eleonora, Gaia, Mary, Stefano, Tommaso
Che ci siete fin dai tempi dell'infanzia e siete cresciuti insieme a me. Avete visto ogni mia sfaccettatura in questi decenni, condividendo le mie parti migliori, i miei difetti e tutte le tappe della mia vita. Voi non siete soltanto i miei amici più profondi, ma una parte fondamentale di chi sono e delle mie radici: sapere che sarete sempre al mio fianco mi dona una serenità impagabile per il futuro.

A Chiara e Alice
Le mie amiche più fidate. Con voi ho condiviso risate e pianti, abbiamo visto persone entrare e uscire dalla nostra vita, ma voi avete scelto di rimanere fin dall'inizio e continuate a volermi bene. Grazie per il vostro supporto e per il vostro modo di essere piacevolmente strambe: in fondo siamo così, il nostro irripetibile "trio degli strambi".

A Edoardo, Luca e Federico
I miei compagni di viaggio in questo percorso universitario. Avete trasformato le lezioni e i noiosi pomeriggi d'inverno in ateneo in momenti di pura gioia e risate a crepapelle. Per tutti quei pranzi sulle scale rosse fuori dall'università, per tutte le lezioni di elettronica passate a chiacchierare in giardino al sole e per tutte le risate ed i pianti per studiare teoria dei circuiti, campi elettromagnetici, teoria dei sistemi... Grazie.

A Giulia
Compagna di studi e complice di tutti i miei scleri universitari. Se questo traguardo è arrivato, il merito è anche delle nostre infinite colazioni al bar dell'università (di cui siamo diventati ufficialmente i principali finanziatori).

Ad Alberto
Che è sempre stato un punto di riferimento, burbero all'apparenza ma dal cuore gentile; grazie per essersi fidato di me, dandomi la possibilità di esplorare il mondo insieme e di far maturare la mia indipendenza.

A tutti coloro che hanno fatto parte del mio percorso di crescita in questi anni
Per non dimenticare nessuno, il mio grazie più sincero a: Nicolò, Gi, Mich, zia Fede e Alessandro, zia Lara e Filippo, tutti i miei nonni Lorenzo, Anna, Alfredo, Gianna, zia Titta e zio Ermanno, Ricky, Stefania, Antonio, Mirko e Vittoria, Agostino, Erika, Aurora, Elena, Ocsana, Diego, Valeria, Lilla, Alessandro S., Giù, Gindi, Barbara, Giovanni, Dimitri, Nicolò, Francesco.